

第一章 微型计算机概论

1.1 微型计算机简介

1.2 微型计算机组成

1.3 微型计算机工作原理

1.4 计算机运算基础（重难点）

一、容易混淆的几个概念：

微型计算机系统、微型计算机、微处理器



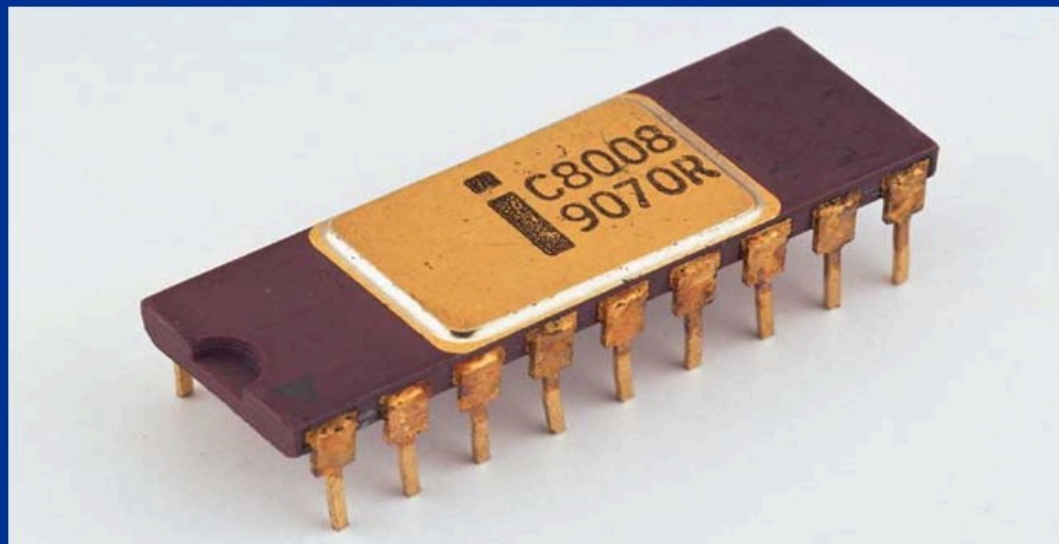
二、微处理器的发展

第一代，1971年开始。4位和低档8位微处理器的时期，其典型产品有：

1971年10月，Intel
4004（4位微处理器）



1972年3月，Intel
8008。集成度 2000个
管 / 片，PMOS工艺，
10 μ m光刻技术。

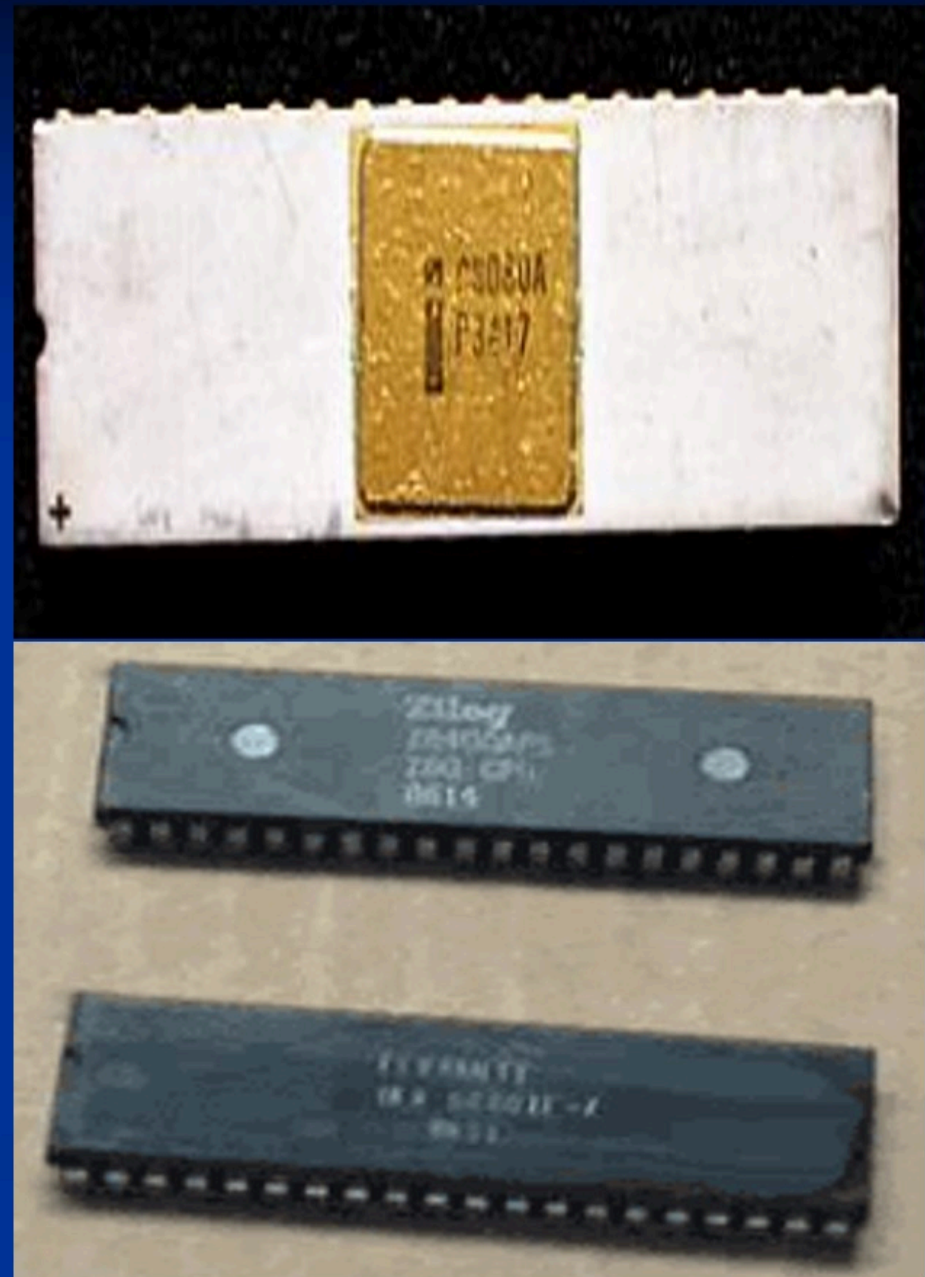


第二代，1973年开始。8位微处理器时期，典型产品有：

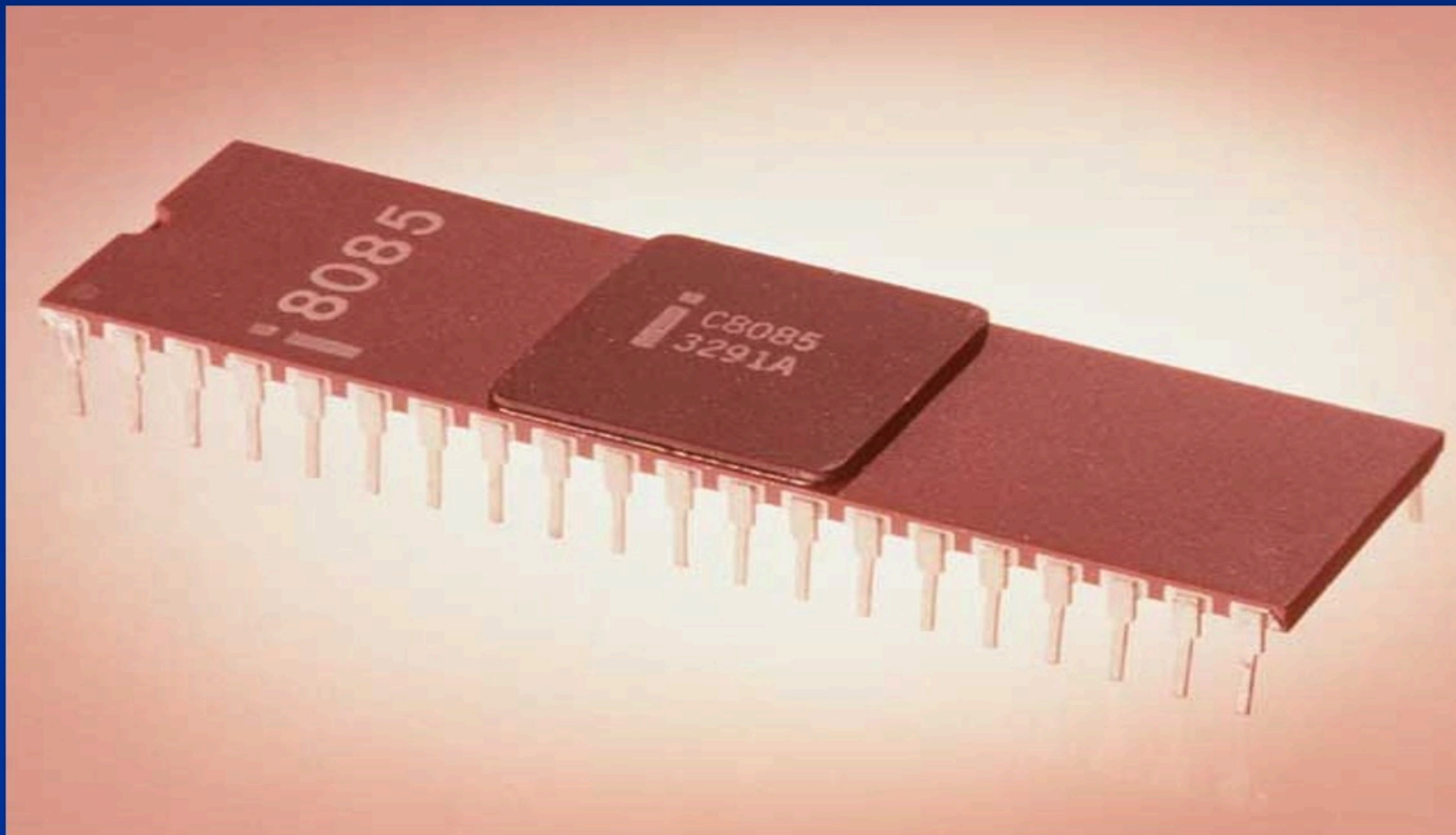
**1973年，Intel:
Intel 8080**

**1974年3月 Motorola:
MC6800; MC6502**

1975、76年 Zilog: Z80 ;



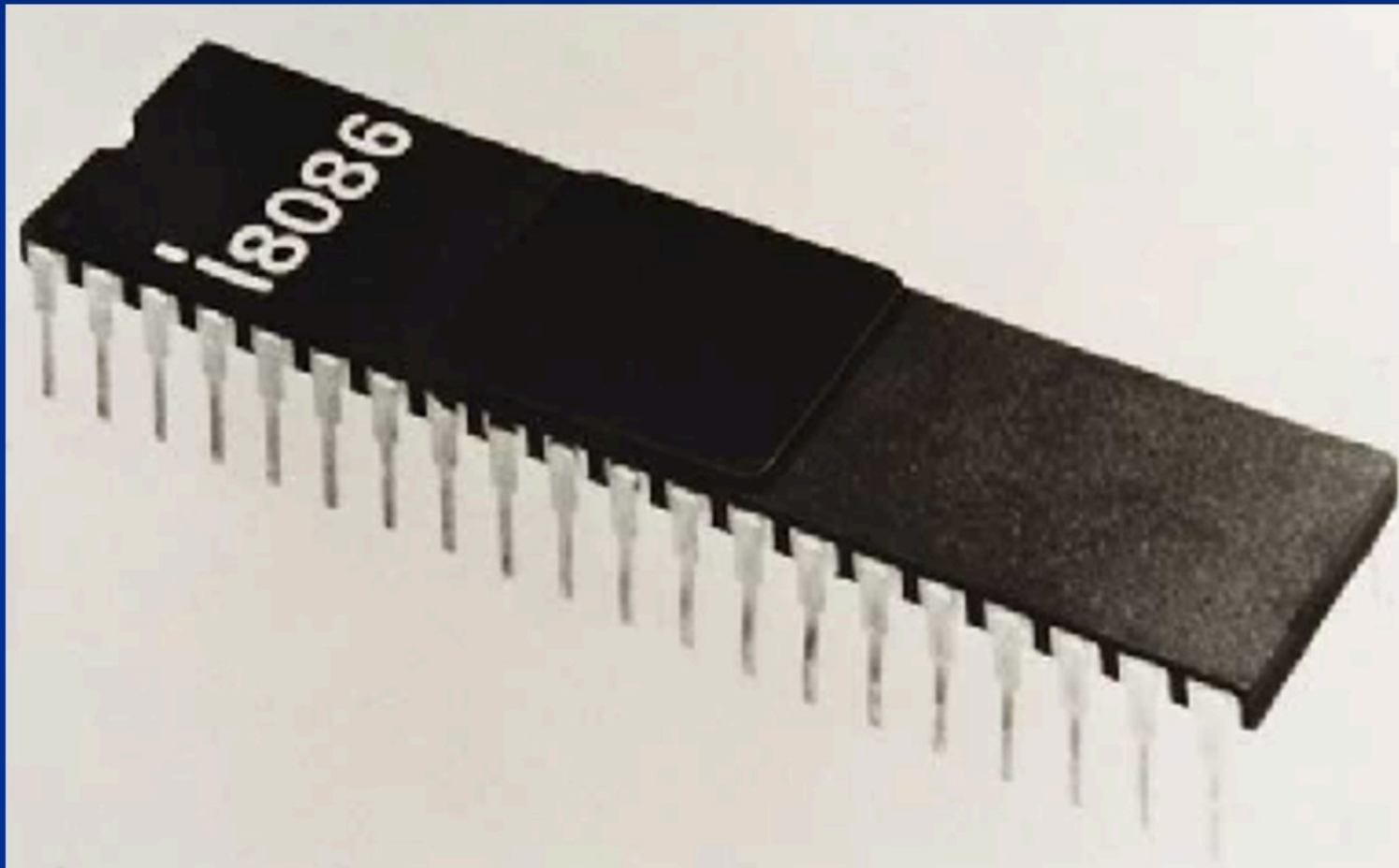
Intel 8085集成度为5400管子 / 片，6 μ m光刻技术。



第三代，1978年开始。16位微处理器时期，其典型产品有：

1978年. Intel 8086

首个16位机、20位地址线、1M地址空间。

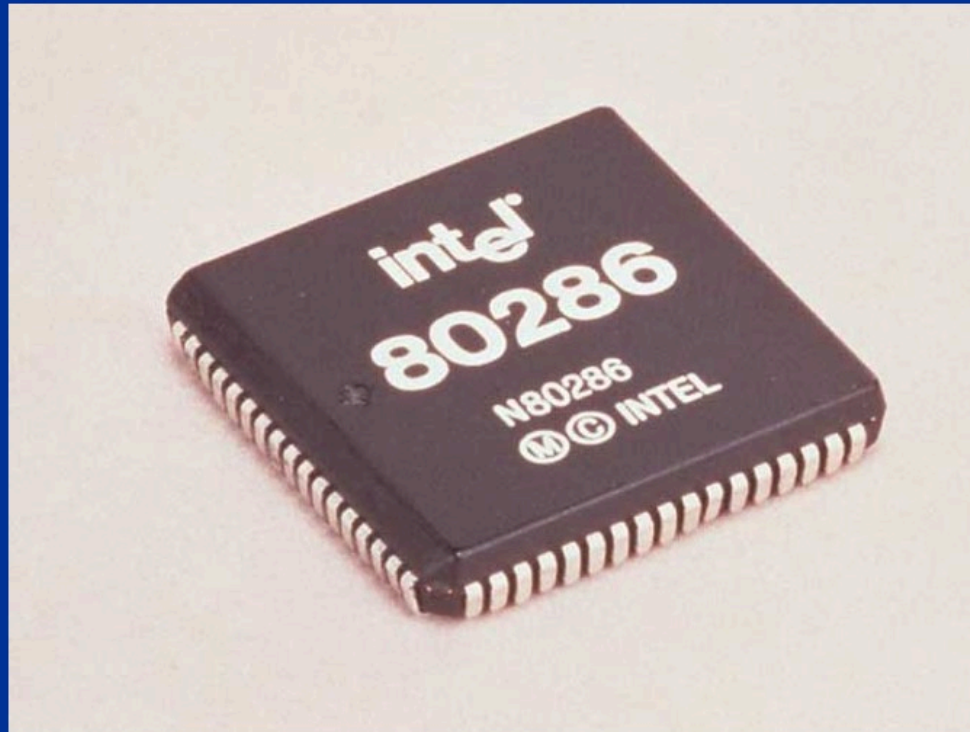


1979年, Zilog: Z8000

Motorola: MC68000

Intel 80286

集成度为68000管子 / 片, 3 μ m光刻技术。



• 数学协处理器**80287**



80287

第四代，1981年开始，32位微处理器时期，其典型产品有：

1983年 Zilog: Z80000

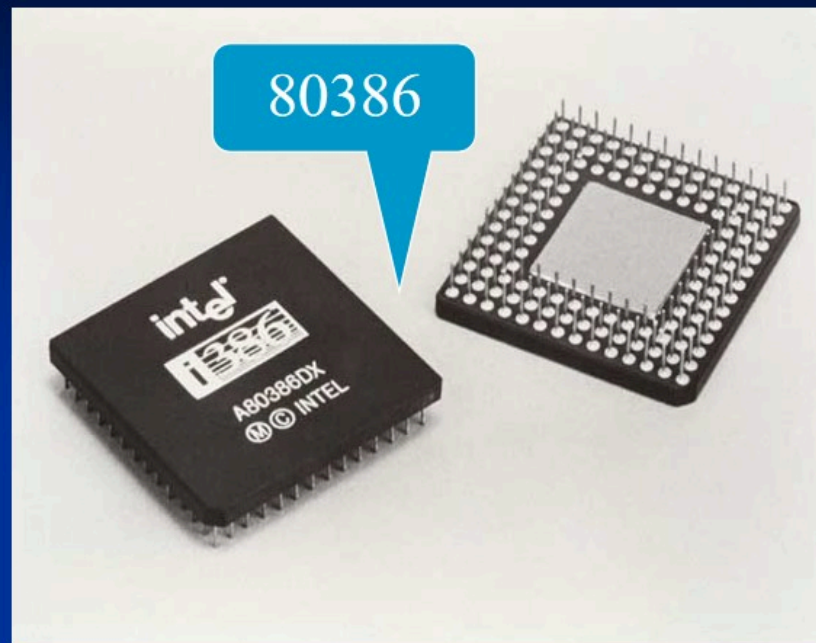
1984年 Motorola:
MC68020

1985年 Intel: Intel
80386,

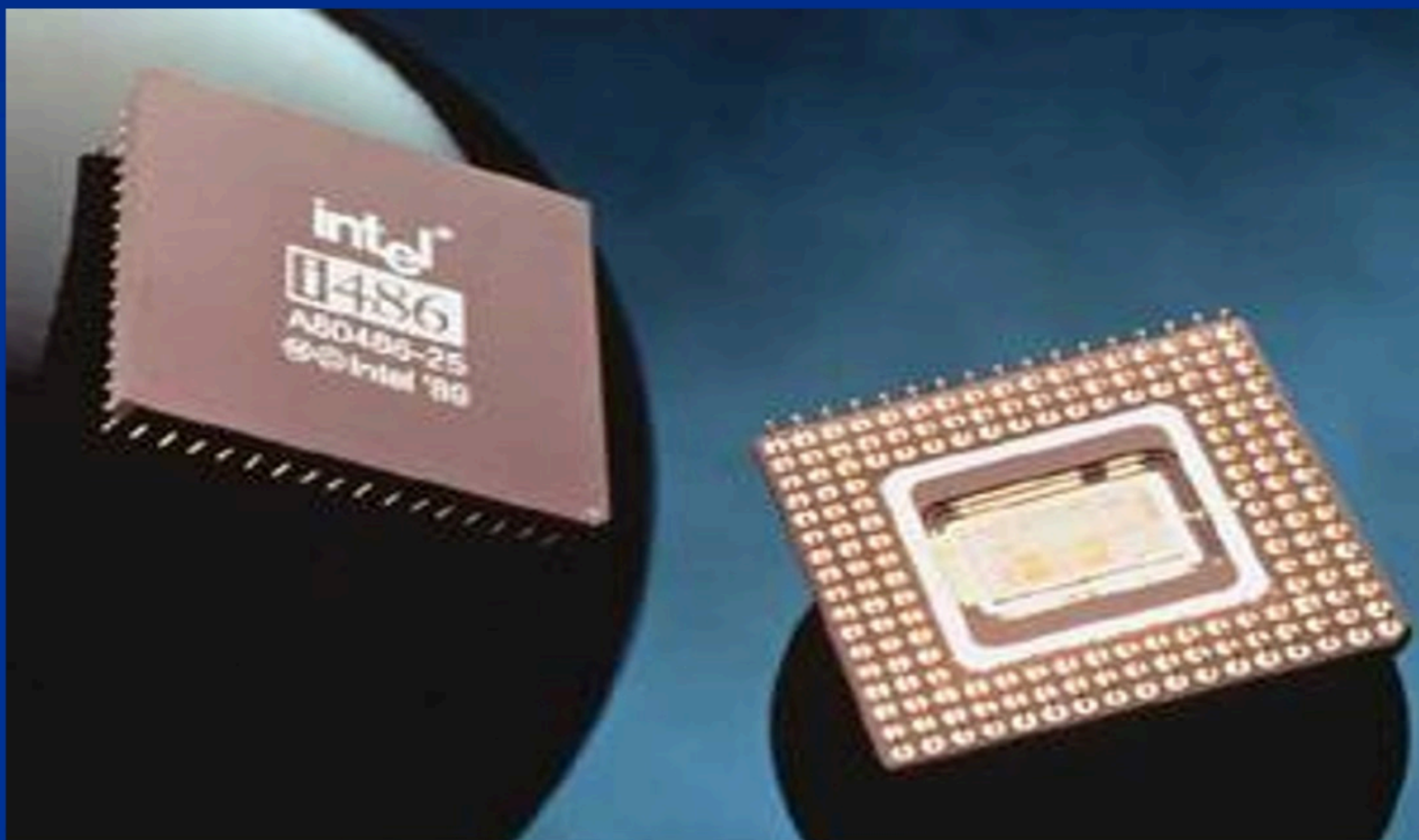
集成度 27.5万管子 / 片，1.2 μ m
光刻技术。

80386后出现了许多高性能的 32
位微处理器。

- 数学协处理器**80387**

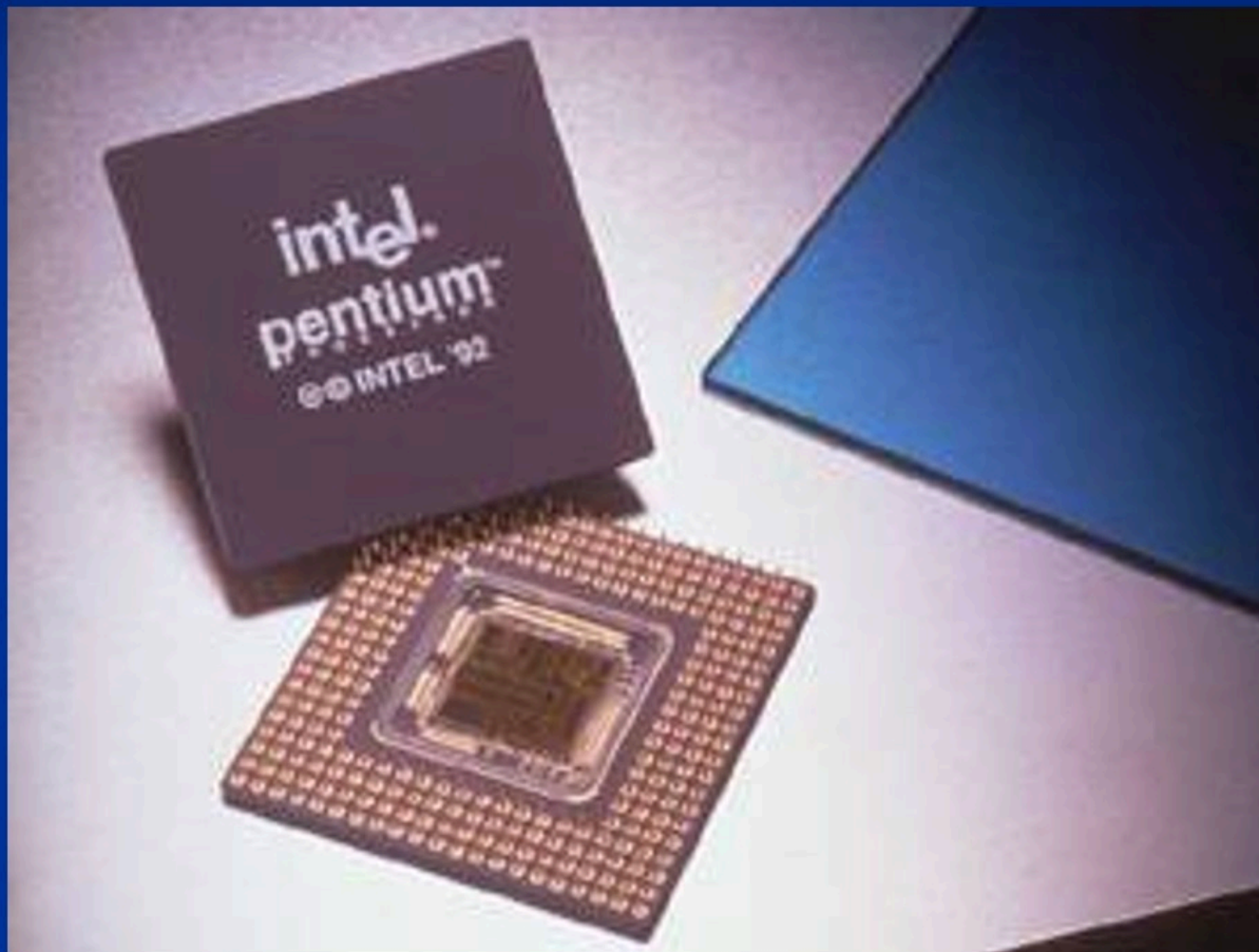


1989年 Intel公司 Intel80486,
1989年 Motorola公司MC68030, MC68040
集成度都已超过100万管子 / 片, 主振频率达25—120MHz。



1993年 Intel Pentium（外部数据总线宽64位）

集成度都已超过100万管子 / 片，主振频率达 200MHz。



32位Intel 80X86系列

1995年, *Pentium Pro* (高能奔腾) 集成度达550万 / 片, 同片 L2Cache256KB。

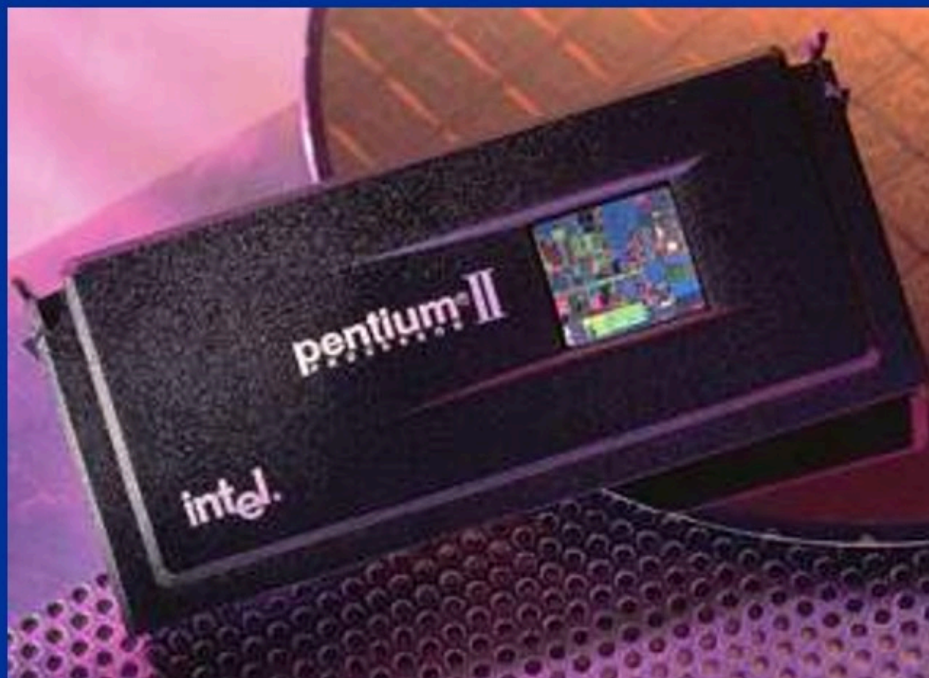
1997年, *Pentium MMX* (多能奔腾、简称 MMX) 。



1997年5月， *Pentium II* (PII) MMX指令集的 *Pentium Pro*， Slot1接口。

片上同频32K LI Cache， 板上1/2主频 512K L2 Cache， 兼容MMX， 外频 100M。

1999年3月， *Pentium III*， 片上同频32K LI Cache和 512K L2 Cache， 兼容MMX， 新增70条指令SSE（Streaming SIMD Extensions）， 其外频为 100M， 并向 133MHz外频发展。



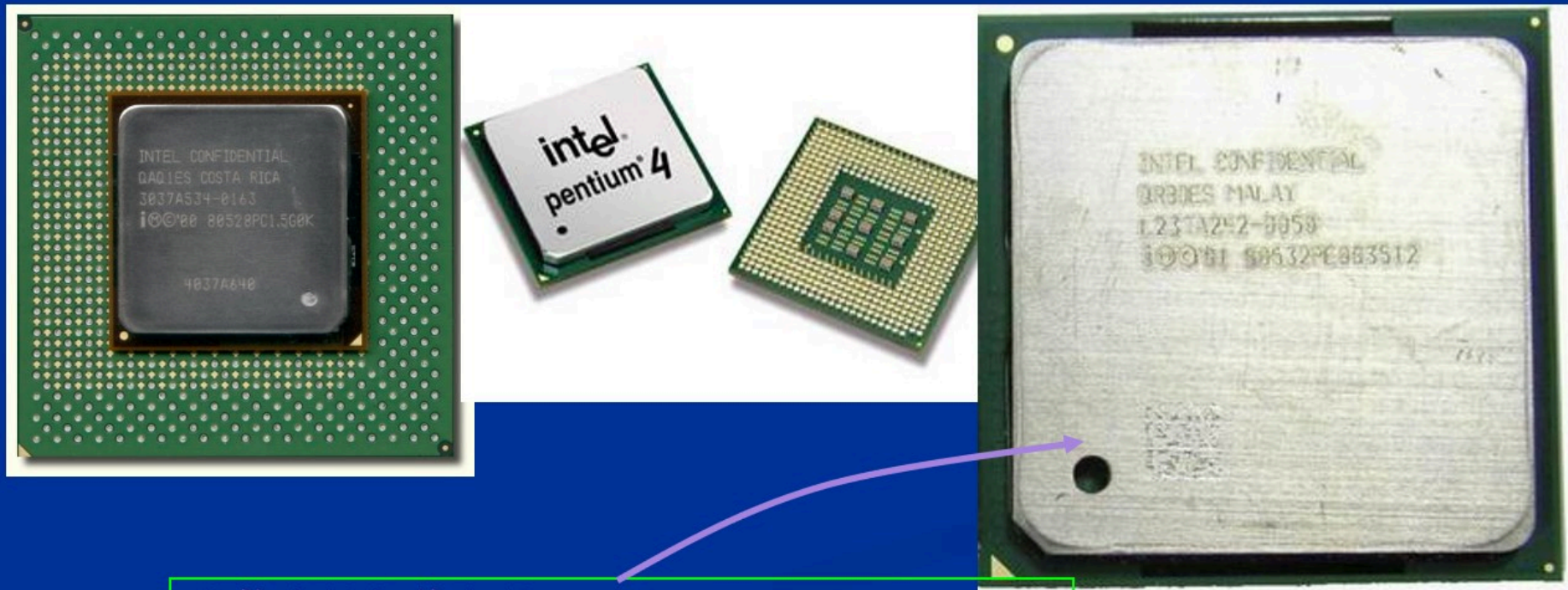
Old Celeron

Celeron

2001年, *Pentium IV* (奔腾IV, P4) 主时钟达2000MHz

2002年, P4主时钟已达2800MHz。

2003年, 3.06GHz的P4, 采用了超线程 (HT) 技术, 在一颗物理核心中整合了两颗逻辑核心, 大大提高处理器的效率。在高负载的多任务应用中能显出其效能。

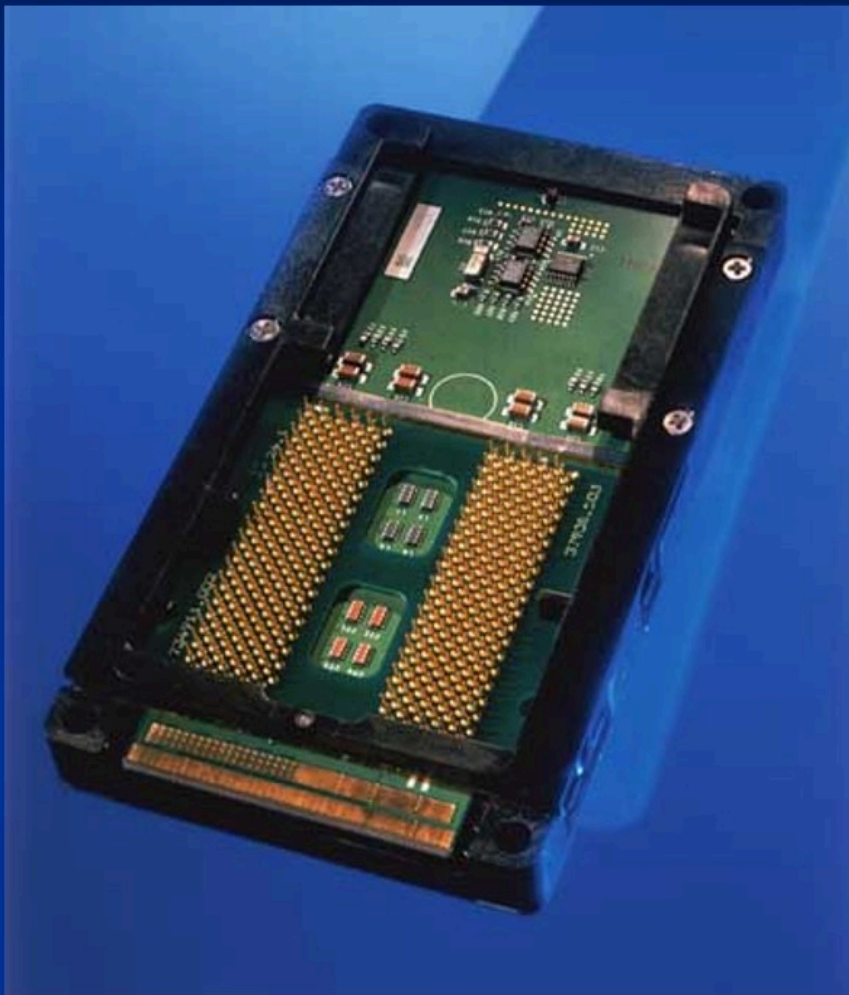


首款支持超线程处理器: Pentium 4 3.06GHz

第五代：2001年开始，64位微处理器出现并得到应用。

Itanium(安腾)

Intel的第一款64位处理器



0.09微米光刻工艺，三级缓存，其中一级缓存与二级缓存都是装载于核心内部，容量分别为32KB与96KB。三级缓存容量达到了2MB或4MB。



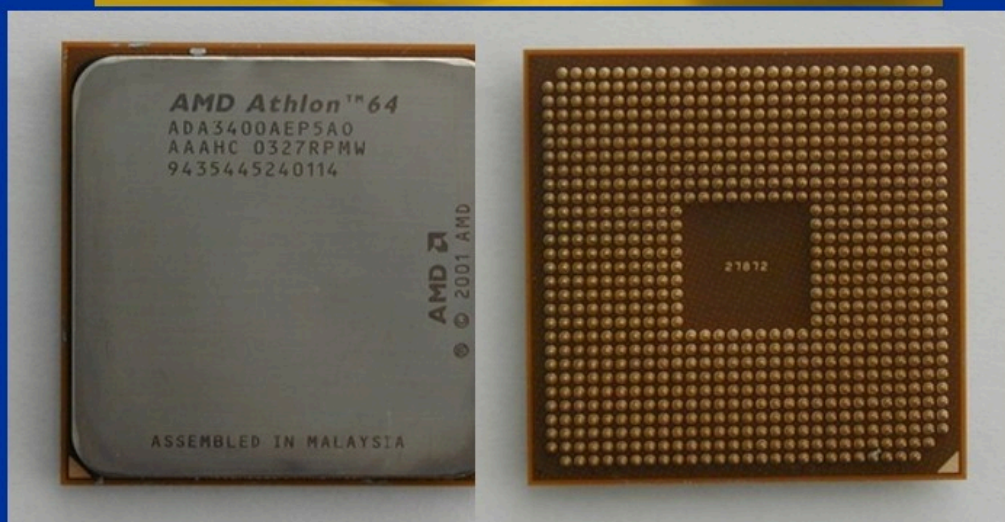
Athlon 64

AMD第一款64位处理器

0.13微米或90纳米光刻工艺

工作频率为2.2GHZ，集成
1MB L2 CACHE，

Socket754封装



三、微机分类方法

- **按字长分类：**即按照微处理器单次处理的数据长度为分类标准，可分为4位，8位，16位，32位，64位微处理器
- **按照系统规模分类：**
单片机，单板机，个人计算机。

- **单片机：**将CPU、RAM、ROM及I/O接口电路等部件集成在一块集成电路芯片上，以芯片形式出现的微型计算机。这是目前使用量最大的一类微型计算机。常见的单片机有Intel 51系列，Atmel AVR系列，Motorola的MC系列等。
- **单板机：**在一个电路板上集成了CPU、存储器、I/O接口电路和一定的输入输出设备，以单块电路板形式出现的微型计算机。常见的单板机有X86架构的PC104系统，ARM架构的单板机系统等。
- **个人计算机：**所谓“个人计算机”（PC Personal Computer），是指“由微处理器芯片装成的、便于搬动而且不需要维护的计算机系统”。通常我们所说的个人计算机是指台式个人计算机和笔记本个人计算机，现在主流的是X86系列个人计算机和苹果公司的MAC系列个人计算机。

第一章 微型计算机概论

1.1 微型计算机简介

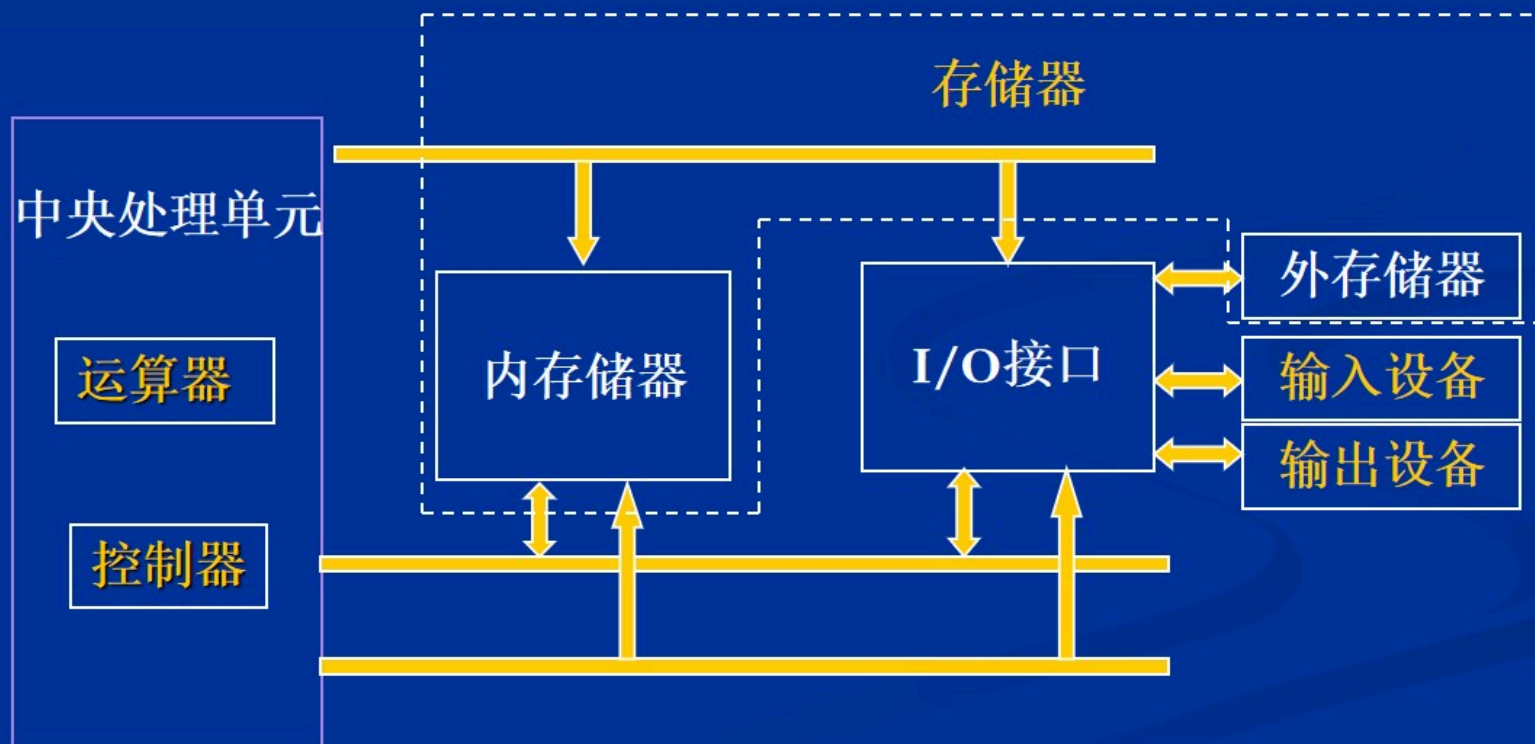
1.2 微型计算机组成

1.3 微型计算机工作原理

1.4 计算机运算基础（重难点）

一、计算机的五大部分：

按照冯·诺依曼体系结构，计算机包括五个主要的部分，即：运算器、控制器、存储器、输入设备和输出设备。



二、微机的三总线结构

在微型计算机中采用总线结构连接微处理器、输入/输出接口、内存储器等部件，它们之间的信息传递通过总线进行。所谓总线，就是计算机中各功能部件间传送信息的公共通道，它是微型计算机的重要组成部分。总线结构使微机具有结构简单，易于维护和扩展等优点，它是微型计算机的一个重要特点。总线按照功能不同一般分为三类：数据总线（DB，Data Bus），地址总线(AB，Address Bus)和控制总线(CB，Control Bus)。

第一章 微型计算机概论

1.1 微型计算机简介

1.2 微型计算机组成

1.3 微型计算机工作原理

1.4 计算机运算基础（重难点）

- 微型计算机是一个由时钟驱动的数字电路系统。微处理器在一个或几个时钟信号驱动下，完成一次操作，一系列的操作完成一定的数据处理任务。

微处理器对指令的执行大致可分为三个步骤:

取指令：微处理器从存储器中将指令读取到微处理器内部，读取指令首先要确定指令所在的地址，这个地址由微处理器内部的指令指针寄存器存放。

分析指令：也称为指令译码，微处理器通过分析读取的指令了解指令的功能和操作对象等，从而为下一步执行做好准备。

执行指令：根据指令分析的结果，微处理器发出一系列控制信号，指挥各部件完成该指令的功能，这样一条指令的执行就结束了。接着微处理器读取下一条指令进入下一个指令周期。

如有两个数求和的指令序列如下：

MOV AL, [2000H]

ADD AL, 10H

OUT 20H, AL

该指令序列在内存中存放情况如下：

2000: 0000 **MOV AL, [2000H]**

2000: 0003 **ADD AL, 10H**

2000: 0005 **OUT 20H, AL**

1. 指令指针寄存器初始值为CS: IP=2000H: 0000H, 指向第一条指令。微处理器首先根据该地址读取第一条指令**MOV AL, [2000H]**，并将IP自动加3变为2003H。

2. 微处理器对读得的指令进行分析译码

3. 分析译码后根据指令功能，读取DS: 2000H地址处的一个字节数据到AL寄存器。

- 第一条指令完成后读取下一条指令，此时指令指针为CS: IP=2000H: 0003H。

读取的指令为**ADD AL,10H**，IP自动加2变为2005H

该指令被分析译码

交给运算器进行加法运算并将结果存在AL寄存器中。

- 第二条指令完成后读取第三条指令，此时指令指针为CS: IP=2000H: 0005H。

读取的指令为**OUT 20H, AL**

该指令被分析译码

微处理器将AL的数据传送给20H端口。

第一章 微型计算机概论

1.1 微型计算机简介

1.2 微型计算机组成

1.3 微型计算机工作原理

1.4 计算机运算基础（重难点）

一、计算机中的数制

(从程序设计者的角度理解“数”)

1、数的位置表示法

用一组符号表示数值时，数值的大小不但与每一个符号所表示的值有关，还与这个符号所处的位置有关，这种表示方式称为数的位置表示法。

例：

$$12.34 = 1 \times 10^1 + 2 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2}$$

$$(1101)_2 = 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0$$

$$(9187)_{16} = 9 \times 16^3 + 1 \times 16^2 + 8 \times 16^1 + 7 \times 16^0$$

数 制	基 数	数 码
二进制 Binary(B)	2	0,1
八进制 Octal (O)	8	0,1,2,3,4,5,6,7
十进制 Decimal (D)	10	0,1,2,3,4,5,6,7,8,9
十六进制 Hexadecimal(H)	16	0,1,2,3,4,5,6,7,8,9, A,B,C,D,E,F

2、数制之间的转换

- 二进制 $\rightarrow$ 十进制

规则：按权展开求和

例如：1011B = ? D

二进制的权：

1	1	1	1	1.
16	8	4	2	1.

$$1011B = 1*8+1*2+1*1=11$$

● 十进制 → 二进制

规则：短除法、凑幂法

例如：29D = ?B

二进制的权：

1	1	1	1	1
16	8	4	2	1

$$\begin{array}{r} 2 \overline{) 29} \\ 2 \overline{) 14} \dots\dots\dots 1 \\ 2 \overline{) 7} \dots\dots\dots 0 \\ 2 \overline{) 3} \dots\dots\dots 1 \\ 2 \overline{) 1} \dots\dots\dots 1 \\ \quad 0 \dots\dots 1 \end{array} \left. \begin{array}{l} \text{低} \\ \uparrow \\ \text{高} \end{array} \right\} = 11101\text{B}$$

短除法

$$29\text{D} = 16 + 8 + 4 + 1 = 11101\text{B}$$

凑幂法

- 二进制→十六进制

规则：以小数点为界每四位二进制数为一组，
转换成对应的十六进制符号。

例如：11010110111111B=?H

0011	0101	1011	1111
↓	↓	↓	↓
3	5	B	F

0011010110111111B = 35BFH

- 十六进制→二进制

规则：把每一位十六进制符号转换成四位二进制数。

例如：A19CH=?B

A	1	9	C
↓	↓	↓	↓
1010	0001	1001	1100

A19CH = 1010000110011100B

课堂练习1

请完成以下进制之间的转换

(1) $110010B = [\text{填空1}] D$

(2) $80 = [\text{填空2}] B$

(3) $1011101110001100011B = [\text{填空3}] H$

(4) $8B6AH = [\text{填空4}] B$

说明：答案中不要包含空格

课堂练习1

请完成以下进制之间的转换

(1) $110010B = ?D$

$$110010B = 32 + 16 + 2 = 50$$

(2) $80 = ?B$

$$80 = 64 + 16 = 1010000B$$

(3) $1011101110001100011B = ?H$

$$101\ 1101\ 1100\ 0110\ 0011B = 5DC63H$$

(4) $8B6AH = ?B$

$$8B6AH = 1000\ 1011\ 0110\ 1010B$$

不同进制数据，相同的表示会有不同的数值大小，为了在表示数据时不产生歧义，通常在数据最后加一个字母来区别不同的进制。

十进制数使用**D**表示（可以省略），二进制数使用**B**表示，八进制数使用**Q**表示，十六进制使用**H**表示。其对照关系如右表所示。

十进制	二进制	八进制	十六进制
0	0000B	0Q	0H
1	0001B	1Q	1H
2	0010B	2Q	2H
3	0011B	3Q	3H
4	0100B	4Q	4H
5	0101B	5Q	5H
6	0110B	6Q	6H
7	0111B	7Q	7H
8	1000B	10Q	8H
9	1001B	11Q	9H

二、计算机中数的表示和运算

（从计算机的角度来理解“数”）

在计算机中无论是数据还是其它信息都是采用一定长度的二进制数表达。对于不同的信息，所需要的二进制数长度也不同。

人们规定：在微型计算机中以八位二进制数为一个基本单位来分配存储地址，称为字节（**Byte**）。任何一个信息最少要用一个八位二进制数来表达，如果需要更长的二进制数表达某些信息则需要使用八的整数倍位二进制数，如**16位**，**32位**等。

1、计算机中多字节数据的存储

地址	数据	地址	数据
100H	12H	100H	78H
101H	34H	101H	56H
102H	56H	102H	34H
103H	78H	103H	12H
104H		104H	

(a) 大端存储

(b) 小端存储

高高高低低

问题1: 4C5DH怎么存放在101H开始的两个存储单元中?

地址	数据	地址	数据
100H		100H	
101H	4CH	101H	5DH
102H	5DH	102H	4CH
103H		103H	
104H		104H	

(a) 大端存储

(b) 小端存储

问题2: 1200怎么存放在101H开始的两个存储单元中?

地址	数据	地址	数据
100H		100H	
101H	04H	101H	B0H
102H	B0H	102H	04H
103H		103H	
104H		104H	

(a) 大端存储

(b) 小端存储

2、算术运算的基本规则

二进制 逢二进一，借一当二

加法规则

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1+1=0 \text{ (进1)}$$

减法规则

$$0-0=0$$

$$0-1=1 \text{ (借1)}$$

$$1-0=1$$

$$1-1=0$$

十六进制 逢十六进一，借一当十六

$$\begin{array}{r} \text{C } 3 \text{ H} \\ + \text{ } 8 \text{ } 5 \text{ H} \\ \hline \end{array}$$

$$1 \text{ } 4 \text{ } 8 \text{ H}$$

$$\begin{array}{r} 3 \text{ } 5 \text{ H} \\ - 2 \text{ } \text{C} \text{ H} \\ \hline \end{array}$$

$$0 \text{ } 9 \text{ H}$$

3、逻辑运算的基本规则

“与”运算 (**AND**)

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

“或”运算 (**OR**)

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

“非”运算
(**NOT**)

A	$\neg A$
0	1
1	0

“异或”运算 (**XOR**)

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

例: $X=00FFH$ $Y=5555H$, 求 $Z=X \vee Y=?$

$X= 0000 \ 0000 \ 1111 \ 1111 \ B$

$\vee \ Y= 0101 \ 0101 \ 0101 \ 0101 \ B$

$Z= 0101 \ 0101 \ 1010 \ 1010 \ B$

$\therefore Z=55AAH$

课堂练习2

完成以下运算

(1) $35H + 8CH =$ [填空1]

(2) $35H - 8CH =$ [填空2]

(3) $35H \text{ AND } 8CH =$ [填空3]

(4) $35H \text{ OR } 8CH =$ [填空4]

(5) $35H \text{ XOR } 8CH =$ [填空5]

(6) $\text{NOT } 8CH =$ [填空6]

说明：答案填十六进制数

课堂练习2

完成以下运算

35H= 0 0 1 1 0 1 0 1B

8CH=1 0 0 0 1 1 0 0B

(1) 35H + 8CH= **0C1H**

(2) 35H - 8CH= **0A9H**

(2) 35H AND 8CH= **04H**

(2) 35H OR 8CH= **0BDH**

(2) 35H XOR 8CH= **0B9H**

(2) NOT 8CH= **73H**

4、有符号数表示

有符号数在计算机中表示时，除了数值大小外，还有相应的正负号，通常正负号以二进制数据最高位来表示。

不同计算机处理能力和表示范围有所不同，表示一个数据所使用的二进制位数也不相同，通常是字节的整数倍，如：8位、16位、32位等。

假设机器字长为8位



符号位=0 表示正数
符号位=1 表示负数

假设机器字长为16位



常用表示法——原码 反码 补码

原码表示法：符号 + 绝对值

例：n=8bit

$$[+3]_{\text{原码}} = 0\ 000,0011 = 03\text{H}$$

$$[-3]_{\text{原码}} = 1\ 000,0011 = 83\text{H}$$

$$[+0]_{\text{原码}} = 0\ 000,0000 = 00\text{H}$$

$$[-0]_{\text{原码}} = 1\ 000,0000 = 80\text{H}$$

0的表示不唯一

反码表示法：正数的反码同原码，负数的反码数值位与原码相反

例：n=8bit

$$[+5]_{\text{反码}} = 0\ 000,0101 = 05\text{H}$$

$$[-5]_{\text{反码}} = 1\ 111,1010 = \text{FAH}$$

$$[+0]_{\text{反码}} = 0\ 000,0000 = 00\text{H}$$

$$[-0]_{\text{反码}} = 1\ 111,1111 = \text{FFH}$$

0的表示不唯一

补码表示法:

正数的补码: 同原码

$$[+1]_{\text{补码}} = 0000\ 0001 = 01\text{H}$$

$$[+127]_{\text{补码}} = 0111\ 1111 = 7\text{FH}$$

$$[+0]_{\text{补码}} = 0000\ 0000 = 00\text{H}$$

负数的补码: (1) 写出与该负数相对应的正数的补码
(2) 按位求反
(3) 末位加一

例: 机器字长8位, $[-46]_{\text{补码}} = ?$

$$[-46]_{\text{原码}} = 1010\ 1110$$

$$\begin{array}{l} \text{按位求反} \\ 1101\ 0001 \\ \text{末位加一} \end{array} \rightarrow 1101\ 0010 = \text{D2H}$$

$$\text{机器字长16位, } [-46]_{\text{补码}} = \text{FFD2H}$$

$$[-1]_{\text{补码}} = 1111\ 1111 = \text{FFH}$$

$$[-127]_{\text{补码}} = 1000\ 0001 = 81\text{H}$$

$$[-128]_{\text{补码}} = 1000\ 0000 = 80\text{H}$$

$$[-0]_{\text{补码}} = 0000\ 0000 = 00\text{H} \quad (+0、-0\text{相同})$$

n位补码的表数范围： $-2^{n-1} \leq N \leq 2^{n-1}-1$

$$n=8 \quad -128 \leq N \leq 127$$

$$n=16 \quad -32768 \leq N \leq 32767$$

课堂练习3

计算以下数据的8位二进制补码

(1) 34

[填空1]

(2) -100

[填空2]

说明：结果填二进制数

课堂练习3

计算以下数据的8位二进制补码

(1) 34

$$[34]_{\text{补}} = 00100010\text{B}$$

(2) -100

$$[-100]_{\text{原}} = 11100100\text{B}$$

$$[-100]_{\text{反}} = 10011011\text{B}$$

$$[-100]_{\text{补}} = 10011100\text{B}$$

5、有符号数的加减运算

计算机中数据的加 / 减运算，通常以补码方式进行：

加法规则： $[X+Y]_{\text{补码}} = [X]_{\text{补码}} + [Y]_{\text{补码}}$

减法规则： $[X-Y]_{\text{补码}} = [X]_{\text{补码}} + [-Y]_{\text{补码}}$

补码进行减法运算可转换为对应的加法运算，计算中符号位参与运算，并能得到正确结果。

6、BCD码

如果使用二进制数对十进制的数码进行编码，并利用编码的位置关系来表达十进制数的位权关系，则可以更方便地在计算机中对十进制数进行表示和运算。常用的编码称为二~十进制码或称BCD码（Binary Coded Decimal）。

如:十进制数100如果直接进行数值转换用二进制数表示就应该是**0110,0100B**（64H）,需要进行7次除2取余计算得到。

而使用BCD编码，就可以直接表示为**0001,0000,0000B**,

1 0 0

也就是直接将十进制数的每一位数码转换成一个对应编号，然后按其位置进行排列来表示该数。这极大地方便了十进制数的表达和转换。

由于计算机中数据地址分配的基本单位是字节，因此，对每一个数码进行编号通常是以字节为单位进行的，也就是说每一个数码将由八位二进制编码。这种BCD码称为非压缩BCD码。

显然非压缩BCD码的浪费很大。如果每一个十进制数码用四位二进制编码，则可以减少一半存储空间。

其对照表如右图：

十进制	非压缩BCD码	压缩BCD码
0	00000000	0000
1	00000001	0001
2	00000010	0010
3	00000011	0011
4	00000100	0100
5	00000101	0101
6	00000110	0110
7	00000111	0111
8	00001000	1000
9	00001001	1001

三、定点与浮点数表示法

定点表示法

定点表示法，是指将所有数据的小数点位置固定，以便于存取和计算。通常将小数点位置固定到纯小数或整数的形式，即把数缩小成纯小数然后记录小数点后尾数的值或把数放大成整数然后记录小数点前面的值。使用时再将其进行放大或缩小相应数量级进行运算。

符号位	数值位
-----	-----

↑ 小数点位置

纯小数形式

符号位	数值位
-----	-----

小数点位置 ↑

纯整数形式

浮点表示法

浮点表示法，是指小数点在数中的位置是浮动的。浮点表示法中以纯小数的形式表示数据的有效位，用浮动的比例因子表示数据的实际大小。任意二进制数 x 总可表示成如下形式：

$$x = \pm d \times 2^{\pm p}$$

阶符	阶码 (p)	数符	尾数 (d)
1位	m位	1位	n位

$2^{\pm p}$ 是比例因子，数据中小数点的位置随 p 的大小处于浮动状态，因此这种表示方法被称为浮点表示法。